

Design a Distributed Cache

You are asked to design and implement a **distributed cache system** that supports the following operations:

- `get(key)`
- `put(key, value)`

Functional Requirements

1. The cache is distributed across multiple cache nodes.
2. The number of cache nodes should be configurable.
3. When `get(key)` is called:
 - If the key is present in the cache, return the value from the cache.
 - If the key is not present in the cache, fetch the value from the database, store it in the cache, and then return it.
4. When `put(key, value)` is called:
 - The value should be stored in the appropriate cache node based on the distribution strategy.
 - You may assume the database is also updated as needed, or clearly mention your assumption.

Design Requirements

Your design should be flexible and extensible.

1. Pluggable Distribution Strategy

The system should support different strategies to decide **which cache node will store a given key**.

For the current implementation, students may use a simple approach such as:

- modulo-based distribution (`hash(key) % numberOfNodes`)
- map-based routing

However, the design should allow plugging in a different strategy in the future, such as:

- **consistent hashing**

2. Pluggable Eviction Policy

Each cache node has limited capacity.

For the current version, the cache must use:

- **LRU (Least Recently Used)** eviction

But the design should allow switching to other eviction policies in the future, such as:

- **MRU (Most Recently Used)**
- **LFU (Least Frequently Used)**

Expectations from the Candidate

Your solution should include:

1. **Class diagram**
2. **Code**
3. Clear explanation of:
 - how data is distributed across nodes
 - how cache miss is handled
 - how eviction works
 - how the design supports future extensibility

Additional Notes / Assumptions

- You may assume keys are unique.
- You may assume the database interface is already available.
- You do not need to implement real network communication between nodes; this is an in-memory LLD exercise.
- Focus on clean object-oriented design and extensibility.